

Full Stack Development Documentation

1. Introduction

Project Title: House Rent MERN - A Comprehensive Full Stack Property Rental Platform

Team Members:

- **PUTTA ROHITH KUMAR GOUD** - Full Stack Developer & Project Lead
 - Role: Architecture Design, Frontend Development, Backend Development, Database Design, API Integration, UI/UX Implementation
 - Responsibilities: End-to-end development of the entire application stack, feature implementation, quality assurance, and project coordination

Project Description: House Rent MERN is a full-stack rental property platform built with the MERN stack (MongoDB, Express.js, React, Node.js). The web application connects property owners, tenants, and administrators on a single marketplace. Property owners can list rental units with full details, tenants can search and book available listings, and admins can verify owners and manage the platform.

2. Project Overview

Purpose

The primary purpose of House Rent MERN is to streamline property rental discovery, listing, and booking. It replaces manual rental workflows with a modern digital experience that supports:

- property owner registration and listing management
- tenant search, filtering, and booking requests
- admin approval and platform monitoring
- secure authentication and role-based access control

Key Features

- Multi-role user system: tenant, property owner, and admin
 - Secure registration and login with JWT and bcryptjs
 - Owner approval workflow for verified property management
 - Property CRUD operations (create, read, update, delete)
 - Search and filtering by location, property type, rent range, and status
 - Booking system with tenant requests and owner approval
 - Admin dashboard for user and property management
 - Image uploads served from the server `uploads` directory
 - Responsive React frontend powered by Vite, Bootstrap, and Material UI
-

3. Architecture

Frontend Architecture

The frontend is built with React and Vite.

Technology stack:

- React 18.2.0
- Vite 5.4.1

- React Router DOM 6.16.0
- Axios 1.6.5
- Bootstrap 5.3.2
- Material UI 5.16.4
- Emotion 11.12.0

Frontend structure:

- `src/App.jsx` - Main application component and route definitions
- `src/main.jsx` - React application entry point
- `src/api.js` - Axios instance configured with backend base URL and auth header interceptor
- `src/modules/common/` - shared pages: `Home` , `Login` , `Register` , `ForgotPassword` , `Toast`
- `src/modules/admin/AdminHome.jsx` - admin dashboard view
- `src/modules/user/owner/` - owner views: `OwnerHome` , `AddProperty` , `AllProperties`
- `src/modules/user/renter/` - tenant views: `RenterHome` , `AllProperties`
- `src/modules/user/PropertyDetail.jsx` - detailed single property page

Frontend behavior:

- User authentication state is stored in `localStorage` under `houserent_auth`
- Protected routes are enforced by role checks and redirects in React Router
- Authenticated requests automatically include the JWT token in the `Authorization` header

Backend Architecture

The backend is built with Express.js and MongoDB.

Technology stack:

- Node.js
- Express.js 4.18.2
- MongoDB
- Mongoose 7.5.2
- dotenv 16.3.1
- bcryptjs 2.4.3
- jsonwebtoken 9.0.1
- cors 2.8.5
- nodemon 3.0.1 (development)

Backend structure:

- `server.js` - Express application setup, middleware, routes, and server startup
- `config/connect.js` - MongoDB connection logic, admin creation, and environment config
- `routes/` - route definitions for auth, properties, bookings, owner, admin, and user endpoints
- `controllers/` - request handlers for auth, properties, bookings, and admin operations
- `models/` - Mongoose schemas for `User` , `Property` , `Booking` , and `BookingHistory`
- `middlewares/authMiddleware.js` - JWT auth and role-based access control
- `uploads/` - static directory for property image files

Database

The application uses MongoDB with the following core schemas:

- `User` - stores name, email, phone, password, role, `currentLocation`, and approval status
- `Property` - stores owner reference, title, description, location, rent amount, property type, furnishing status, amenities, images, and availability status

- `Booking` - stores property reference, tenant reference, start/end dates, and booking status
- `BookingHistory` - tracks actions related to bookings for future auditing

Admin user is created automatically at database connection if `ADMIN_EMAIL` and `ADMIN_PASSWORD` environment variables are set.

4. Setup Instructions

Prerequisites

- Node.js installed
- npm installed
- MongoDB installed and running
- Environment variables configured in a `.env` file

Environment Variables

Create a `.env` file in `server/` with:

- `MONGO_URI` - MongoDB connection string
- `JWT_SECRET` - secret key for JWT signing
- `PORT` - optional server port (defaults to 8000)
- `ADMIN_EMAIL` - admin account email used during initial setup
- `ADMIN_PASSWORD` - admin account password used during initial setup

Installation

1. Clone the repository.
 2. Install backend dependencies:
 - `cd /home/prko/Documents/Project_mern/server`
 - `npm install`
 3. Install frontend dependencies:
 - `cd /home/prko/Documents/Project_mern/client`
 - `npm install`
 4. Start MongoDB.
-

5. Folder Structure

Client

`client/`

- `index.html` - Vite HTML entry file
- `package.json` - frontend dependencies and scripts
- `vite.config.js` - Vite configuration
- `src/`
 - `main.jsx` - app bootstrap
 - `App.jsx` - routes and auth state
 - `api.js` - Axios client
 - `modules/`
 - `common/` - shared UI components and pages
 - `admin/` - admin dashboard
 - `user/owner/` - owner-specific property management pages

- `user/renter/` - renter property browsing pages

Server

`server/`

- `server.js` - main Express server
 - `package.json` - backend dependencies and scripts
 - `config/`
 - `connect.js` - database connection and admin user creation
 - `db.js` - exports MongoDB connection logic
 - `controllers/` - business logic for each route
 - `models/` - Mongoose schemas
 - `middlewares/` - authentication middleware
 - `routes/` - API route definitions
 - `uploads/` - property image uploads
-

6. Running the Application

Frontend

From `client/` :

- `npm run dev`

Backend

From `server/` :

- `npm start` (production)
- `npm run dev` (development with nodemon)

Access

- Frontend: `http://localhost:5173` (default Vite port)
 - Backend API: `http://localhost:8000`
-

7. API Documentation

Authentication

- `POST /api/auth/register`
 - Request body: `{ name, email, phone, password, role, currentLocation }`
 - Registers tenants and owners
 - Owners require admin approval before managing listings
- `POST /api/auth/login`
 - Request body: `{ email, password }`
 - Returns `{ token, user }`

Properties

- `GET /api/properties`
 - Public endpoint to list properties

- Query filters: `location` , `propertyType` , `status` , `minRent` , `maxRent`
- `GET /api/properties/:id`
 - Fetch property details by ID
- `POST /api/properties`
 - Owner-only
 - Creates a property listing
- `PUT /api/properties/:id`
 - Owner-only
 - Updates an owner's property
- `DELETE /api/properties/:id`
 - Owner-only
 - Deletes an owner's property

Owner Routes

- `GET /api/owner/properties`
 - Owner-only
 - Returns properties owned by the authenticated owner

Booking Routes

- `POST /api/bookings`
 - Tenant-only
 - Request fields: { `propertyId` , `startDate` , `endDate` }
 - Creates a booking request and marks property as booked
- `GET /api/bookings/me`
 - Tenant-only
 - Returns bookings for the logged-in tenant
- `GET /api/bookings/owner`
 - Owner-only
 - Returns bookings for owner properties
- `PUT /api/bookings/:id/status`
 - Owner-only
 - Updates booking status: `pending` , `approved` , `rejected`

Admin Routes

- `GET /api/admin/users`
 - Admin-only
 - Lists all users
- `PUT /api/admin/approve-owner/:id`
 - Admin-only
 - Approves an owner account
- `GET /api/admin/properties`
 - Admin-only
 - Lists all properties

8. Authentication

Authentication Flow

- Users register via `/api/auth/register`
- Passwords are hashed using `bcryptjs`
- Login uses `/api/auth/login` and issues a JWT token valid for 7 days
- Tokens are stored in browser `localStorage` and attached via `Authorization: Bearer <token>` to API requests

Authorization

- `authMiddleware` verifies JWT tokens and loads the `req.user`
- `roleMiddleware` restricts access by role: `tenant`, `owner`, or `admin`
- Owners require `approved: true` to access owner-specific routes

Security

- `bcryptjs` provides password hashing and verification
 - `jsonwebtoken` manages signed tokens
 - `dotenv` protects sensitive configuration values
 - `cors` enables frontend-backend communication
-

9. User Interface

Pages and Flows

- Home page with navigation and platform introduction
- Register page with role selection for tenant or owner
- Login page for all users
- Owner dashboard to manage property listings
- Add Property page for new listings
- Renter dashboard to browse properties and view bookings
- Property detail page with full property information and images
- Admin dashboard for user approval and property monitoring

UI Libraries

- Bootstrap for layout and responsive design
 - Material UI for reusable components and styling
 - Custom React components for app-specific views
-

10. Testing

Current Testing Strategy

- Manual testing of registration, login, property creation, and booking flows
- Verify role-based access control by logging in as tenant, owner, and admin
- Validate CRUD operations on properties and bookings
- Confirm JWT token behavior and protected route enforcement

Recommended Enhancements

- Add unit tests for controllers and middleware
 - Add integration tests for API endpoints using Jest and Supertest
 - Add UI tests for React components using React Testing Library
-

11. Screenshots or Demo

Add screenshots of the following screens when available:

- Home page
 - Login/Registration forms
 - Owner dashboard
 - Property listing page
 - Property detail page
 - Admin dashboard
-

12. Known Issues

- `ForgotPassword.jsx` appears present in the frontend but may not be connected to any active password reset backend endpoint
 - `BookingHistory` model exists in the backend but the current controllers do not create or manage booking history records
 - Owner approval is required before owners can manage listings; unapproved owners cannot access owner pages
 - Some user flows rely on local storage and will reset if browser data is cleared
-

13. Future Enhancements

- Implement full password reset flow with email verification
- Add property image upload support with multipart form data
- Add search by amenities, location radius, and map integration
- Add admin analytics dashboard with charts and metrics
- Implement messaging between tenants and owners
- Add booking cancellation and refund workflows
- Add user profile editing and photo uploads
- Support pagination, sorting, and saved favorites